

Programming Project #3: Implement Traceroute

The socket library you used in project 1 is not limited to TCP. The library supports a range of transport, network, and link layer protocols. In this assignment you will use a feature of the socket library to create customized IPv4 layer datagrams, allowing you to modify the TTL in the IPv4 header, developing your own version of the ***traceroute*** utility.

To develop complete the project you will need to use what are called ***raw*** sockets. On Linux, the ability to read and write to raw sockets is limited to users with superuser (a.k.a. root) level privileges. Because of this you will need to develop your project where you have root access. The Apporto VDI is the recommended development environment. If you choose to develop on a personal device I would encourage you to use a container or a VM – debugging code that runs as root on a system you use for other things is a recipe for disaster.

To test your program, you need only try to your solution from the command line. However, I strongly recommend that you make use of Wireshark to inspect the packets your program sends.

Functional Requirements (CSCI-471 and 598J)

At a minimum, your submission must do the following to receive full credit.

- Basic requirements:
  - The executable should be called traceroute.
  - Your program must be written in C/C++, and must compile and run on Ubuntu 20.04LTS. We will test your submission in the virtual lab.
  - Your program should take one required command line flag/argument, -t and the IP address of the target host.
  - You must provide a verbose mode that provides basic debugging information at each step your program takes. The optional **–d DEBUG_LEVEL** flag should enable the verbose mode.

- Traceroute functionality:
  - Your program needs to send a series of up to 30 ICMP Echo Request datagrams to the destination IP address.
  - The first datagram in the series should have a TTL of 2.
  - The TTL field in the IP header of each successive datagram should be one more than the previous datagram.

- You should stop sending datagrams.
    - When you have sent 30 datagrams.
      OR
    - When you receive and ICMP Echo Response
- After each datagram you send one of three things will happen:
    - You will receive an ICMP Echo Response, at which time your program should print a final message and exit.
    - You will receive and ICMP Timeout; TTL Exceeded datagram, at which time your program should print the IP address of the device that sent the ICMP message and send the next ECHO Request.
    - You will receive no response for at least 15 seconds, at which time you should print the message "No response with TTL of X" and send the next ECHO Request.

Hints and Suggestions.

I have provided submental documentation on Canvas intended to lead you through the steps required for this assignment. You can, and should, also consult online resources for help using raw sockets. You may ask generative AI programs for advice or samples. For example, "How do I create a RAW socket?" or "how do I convert from a string to an 32 bit IP address structure?" would be acceptable questions. You must not directly include any generated code in your submission.

There is no starter code, however the submental documentation includes flow diagram that lays out how I approached the project. You are not required to follow this outline, but I believe it will help.

There is a tarball on Canvas with a makefile, traceroute.h and traceroute.cpp files you may use. The traceroute.cpp file includes a function that will calculate the internet checksum over an arbitrary array that you may use to compute the checksum that goes in the ICMP header.

What to submit.

This assignment is due by the end of the day on Sunday, December 1st. There will be a 1% per day penalty for late submissions.

You should submit a tarball of a single directory containing a makefile, a README.txt with your name and any information I need to compile the program and the source files needed to build your program. The single directory must be named with your username. The grader should not need to do anything other than untar your files, cd into your directory, type make and begin testing.

Do not include any core, object or binary files in the tarball. The Makefile I provided includes a target named **submit** that will create the tarball in the format we are looking for. All you need to do is:

$> make submit

In addition to functionality, you will be graded on the quality of the code, including readability, comments and the use of proper programing practices.

1.0 Introduction

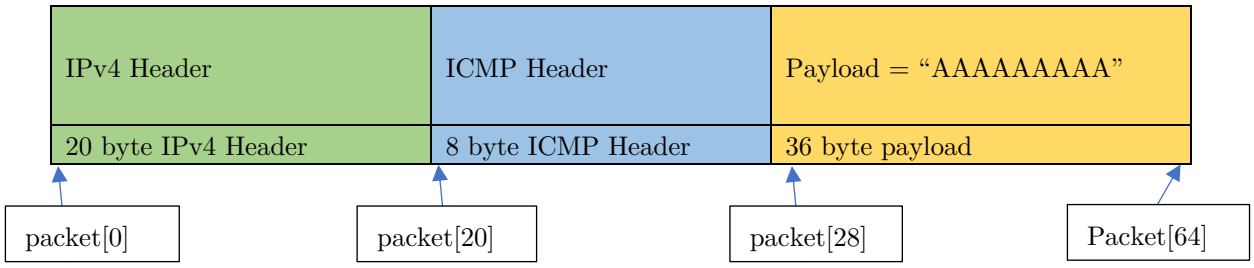
The socket library you used in project 1 is not limited to TCP. The library supports a range of transport, network, and link layer protocols. In this assignment you will use a feature of the socket library to create customized IPv4 layer datagrams, allowing you to modify the TTL in the IPv4 header, developing your own version of the ***traceroute*** utility.

2.0 Building a Datagram in Memory

We know from what we have learned about the network and transport layer, and from looking at Wireshark, that a packet is nothing but a list of bytes. While the transport and network layers usually create the datagram we can build our own datagram in memory as an array of chars. If we had functions that fill in the fields in the IP and ICMP header then the pseudo code might look something like:

```
void buildDatagram () {
  int length = sizeof(ip-header) + sizeof(icmp-header) + sizeof (payload);
  char *packet = new char[length]();
  fill_in_IP_header(packet[0]);
  fill_in_ICMP_header(packet + sizeof(ip-header));
  memset(packet + sizeof(ip-header) + sizeof(icmp-header),'A',sizeof(payload);
}
```

In memory the result would look something like this.



To create the functions that fill in the header, we could use pointer arithmetic and bit-level operators **AND** (a.k.a. &) and **OR** (a.k.a. |). However C/C++ provides a more straightforward approach. The headers `netinet/ip.h` and `netinet/icmp.h` contain a **`struct`** definition for the IP and the ICMP header.

For example, the header file netinet/ip.h contains a struct definition for the IP header that looks like this. Notice the use of bitfields[1] to help with fields in the header that are less than 1 byte long and will automatically deal with byte ordering inside the header.

```
struct iphdr
  {
#if __BYTE_ORDER == __LITTLE_ENDIAN
    unsigned int ihl:4;
    unsigned int version:4;
#elif __BYTE_ORDER == __BIG_ENDIAN
    unsigned int version:4;
    unsigned int ihl:4;
#endif
    uint8_t tos;
    uint16_t tot_len;
    uint16_t id;
    uint16_t frag_off;
    uint8_t ttl;
    uint8_t protocol;
    uint16_t check;
    uint32_t saddr;
    uint32_t daddr;
    /*The options start here. */
  };
```

We can use these structures to help us place values where they need to be. For example, to fill in the IP version, header length, and Type of Service fields the code might look like this:

```
void fill_in_IP_header(char *packet) {

  struct iphdr *ip = (struct iphdr *)packet;         // Cast the pointer

  ip->version = 4;
  ip->ihl = 5;                                       // Header length / 4
  ip->tos = 0;

  // Fill in the rest of the header fields.
}
```

3.0 Place the Datagram on the Network

Once the datagram is built we can use the socket layer to place it on the network. To do so we have to skip over the transport and network layer, giving our datagram directly to the link layer. The mechanism provided by the socket library for talking directly to the link layer is called a RAW socket. The API for a raw socket is the very similar to the API for the TCP socket we used for project 1 and 2. We use an address family of AF_INET, but rather than a stream socket we specify SOCK_RAW.

```
if ((sockFD = socket(AF_INET, SOCK_RAW, IPPROTO_RAW)) == -1) {
   perror("socket");
}
```

---

[1] Bitfields are something you might have only seen very briefly in CSCI 200/261. You can read more about them here: https://www.geeksforgeeks.org/bit-fields-c/

The third parameter in that example warrants some explanation (i.e., IPPROTO_RAW). There are multiple flavors of RAW sockets. Some fix the network layer protocol but allow the application to define its own transport layer header. For example, using IPPROTO_IP would create a raw socket that uses IPv4, allowing you the programmer to write a TCP, UDP (or some totally different) transport layer header. IPPROTO_RAW creates a socket that allows the programmer to write their own IP header, but it must be an IP header. Using IPPROTO_RAW you couldn't create a totally new network layer header. One advantage of IPPROTO_RAW is that we are free to fill in most of the fields in the IP header but the kernel will fill in the ID, Source IP and Checksum fields.

Once we have built the packet and created the socket we can send the socket using the `sendto` function.

```
if (sendto(sendFd, packet, length, 0, (struct sockaddr *) &dest_addr, sizeof(dest_addr)) == -1)
{
    perror("sendto");
    return -1;
}
```

Notice that even though we fill in the destination address in the packet header we still must pass the destination address so that the operating system knows which interface to place the packet on.

4.0 Reading an ICMP Segment from Network

In this assignment we will be placing ICMP Echo Request packets on the network and checking for either an ICMP Echo Reply or ICMP TTL exceeded packet in response. ICMP is not connection oriented; we can't read from the socket we created to send our packets. We need a second socket to read from, this time we will use IPPROTO_ICMP to indicate that we want a RAW socket that can read ICMP packets.

```
if ((recvFd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)) == -1) {
  perror("socket");
  return -1;
}
```

Reading from an ICMP socket is different from reading from a TCP socket. ICMP is not a transport layer protocol which means, among other things, it does not provide the multiplexing/demultiplexing service the way UDP or TCP do. In other words, ICMP packets are delivered to the computer, not to a specific process. This means that when we read from an ICMP socket it will return any and all ICMP packets delivered to the operating system. We must check to see if the ICMP packet we read is the one we are looking for. I will cover the details of how to identify ICMP packets in the next section. The pseudo code for reading an ICMP packet might look like:

```
if (recvfrom(recvFd, recv_buf, RECV_BUF_SIZE, 0, (struct sockaddr *)&recv_addr, &addr_len) < 0)
{
   perror("recvfrom");
   return -1;
}
```

One nice feature of the recvfrom function call, it will fill the source address so we can easily tell who sent the packet. It places the sending address in the recv_addr parameter, using the sockaddr_in structure format.

5.0 IP Addresses Representation

In dotted decimal notation the IP address of cs.mines.edu is 141.193.213.10. We know this is just one way to represent the 32-bit number that is the address. The binary representation would be 10001101.11000001.11010101.00001010 in decimal it is 2378290442. For this assignment we will need to covert from dotted decimal to binary and back.

We will also need to use an ***address struct*** to pass the IP address into and out from the sendto() and recvfrom() functions

There are three methods we will use:

```
// Convert from a dotted decimal string to network representation:
inet_addr(destIP.c_str());

// Place a dotted decimal string into a address structure
struct sockaddr_in dest_addr;
memset(&dest_addr, 0, sizeof(dest_addr));
dest_addr.sin_family = AF_INET;
dest_addr.sin_addr.s_addr = inet_addr(destIP.c_str());

//Convert an address structure to a dotted decimal string.
inet_ntop(AF_INET, &recv_addr.sin_addr, respondent_ip, INET_ADDRSTRLEN);
```

6.0 Non-Blocking Reads and

The final thing we have to know to successfully write our own version of traceroute is how to confirm that the ICMP packet we read with recvfrom() is a response to the ICMP Echo Request we sent. However, before we can address that question we need to know how to poll the socket to see if there is a packet ready for us to read[2].

To poll the socket to see if there is data ready we can use select(). Select is designed to see if any sockets in a collection of sockets is ready for reading, writing or if it has an exception. The select signature looks like:

```
int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timout);
```

- nfds: length of the file descriptor set. Needs to be set to 1 more than the largest file descriptor you are interested in.
- readfds: The set of file descriptors (a.k.a. sockets) you would like to check to see if they have data you can read.
- writefds: The set of file descriptors you would like to test to see if they can be written to.
- exceptfds: the set of file descriptions you want to check for exceptions.

---

[2] At first glance it may not be obvious why we need to poll the socket, rather than just reading from it. We know that not every router responds with a TTL exceeded packet and even if the router does respond the response could be lost. If we send a packet with sendto()and the router that discards our packet does not respond when we call recvfrom() our program will wait (a.k.a. block) until it gets a packet. If no packet arrives, our program ends up waiting forever. The solution is to confirm that there is a packet ready to be read before we call recvfrom().

- timeout = how long to wait until any of the conditions you are checking for become true.

We use the macro FD_ZERO, FD_SET and FD_ISSET to clear the set of file descriptors, mark a file descriptor in a set, and check to see if a file descriptor is in the set.

- FD_ZERO(&mySet) -> empty the set of file descriptors called mySet.
- FD_SET(fd,&mySet) -> Add the file descriptor "fd" to the set called mySet.
- FD_ISSET(fd,&mySet) -> Return true if the file descriptor "fd" is in mySet, false otherwise.

The following code will wait 5 seconds for a packet to arrive on the socked associated with file descriptor recvFd. If there is no packet after 5 seconds it will print an message and wait another 5 seconds. When a packet finally does arrive it prints a message and exits the while() loop.

```
bool ready = false;
int selectReturned = 0;
fd_set readFdSet;
struct timeval timeout;


while (!ready) {

  // Set timeout to 5 seconds.
  timeout.tv_sec = 5;
  timeout.tv_usec = 0;

  // Set the bit in the set corresponding to file descriptor recvFd
  FD_ZERO(&readFdSet);
  FD_SET(recvFd, &readFdSet);

  // Call select()
  DEBUG << "Waiting up to 5 seconds to see if data becomes avalable" << ENDL;
  if ((selectReturned = select(recvFd + 1, &readFdSet, NULL, NULL, &timeout)) < 0) {
    perror("select");
    return -1;
  }
  DEBUG <<"selectReturned = " << selectReturned << ENDL;


  // Confirm that select returned because our FD is set, not because of a timeout.
  if (FD_ISSET(recvFd,&readFdSet)) {
    DEBUG << "Bit #" << recvFd << " is set" << ENDL;
    ready = true;
  }
}
```

7.0 Identifying ICMP

Using select() we can be sure that when we call readfrom() our program will not hang because there is no data. However, we still don't know that the ICMP packet we received was sent in response to the ICMP Echo Request message we sent. Since ICMP packets are errors there are many possible ICMP messages that we could receive depending on what happened to our request. For this assignment you only need to worry about two possibilities (1) a correct ICMP Echo Response and (2) a TTL Exceeded.

7.1 ICMP Echo Response

You can confirm something is an ICMP echo response by looking in the header of the packet you received when you called recvfrom() and confirming it contains TYPE 0 (ICMP_ECHOREPLY). While you are not required to for this assignment, you can match the Echo Reply with a specific the Echo Request by

setting the id and sequence number in the request. Since those values are copied into the reply you can match them up when the reply arrives.

7.2 ICMP TTL Exceeded

To check to see if a datagram is a TTL exceeded you first must check to see if the type is 11 (ICMP_TIME_EXCEEDED), and then check the subtype is 0 (ICMP_EXC_TTL). You don't have for this assignment, but you can check to see what was in the packet when the TTL went to 0 by checking the payload of the TTL exceeded message. The router that reduces the TTL to zero and sends back the ICMP error will copy the IP header and the first 64 bytes of the original packet in the error.

8.0 Program Flow

1. Allocate two 64 byte buffers. One for sending and one for receiving.
2. Fill the whole buffer with a pattern of characters of your choice.
3. Fill in all the fields of the IP header at the front of the buffer.
    a. You don't need to fill in source IP or checksum
4. Fill in all the fields of the ICMP header right behind the IP header.
5. Create the send and receive sockets.
6. while (CURRENT_TTL <= 31) and (reply-not-received)
    a. Set the TTL in the IP header in the buffer to CURRENT_TTL
    b. Set the checksum in the ICMP header
    c. Send the buffer using sendfrom()
    d. While (now < START_TIME + 15) and (not-done-reading)
        i. Use select() to sleep for up to 5 seconds, wake up if data arrives.
        ii. If data has arrived, read it with recevfrom()
            1. If received data is Echo Reply from the destination
                a. Print message
                b. Set reply-not-received to false
                c. Set not-done-reading to false
            2. If received data is TTL Time Exceeded; TTL
                a. print message
                b. Set not-done-reading to false
    e. Increment TTL by 1.